



Mảng tiền tố (Prefix Sum)

Nội dung

- 1. Bài toán mở đầu: Truy vấn tổng đoạn
 - Cách "thô": Duyệt từng phần tử
 - Câu hỏi: Có cách nào trả lời mỗi truy vấn trong $\mathcal{O}(1)$ không?
- 2. Xây dựng mảng tiền tố
 - Định nghĩa
 - Ví dụ trực quan từng bước
 - Công thức tính tổng đoạn
 - Kiểm tra lại với ví dụ
 - Code
- 3. Prefix Sum với mảng đánh số từ 0
- 4. Ví dụ ứng dụng: Tìm đoạn con có tổng bằng $\mathcal{S}$
- 5. Mảng hiệu (Difference Array)
 - Bài toán mở đầu
 - Ý tưởng Difference Array
 - Ví dụ trực quan
 - Code
- 6. Tổng đoạn con lớn nhất (Kadane's Algorithm)
 - Bài toán
 - Cách "thô": Thử mọi đoạn con — $\mathcal{O}(n^2)$
 - Kadane's Algorithm — $\mathcal{O}(n)$
 - Ví dụ trực quan từng bước
 - Code
- 7. Prefix Sum 2D (mở rộng)
- 8. Sai lầm thường gặp
 - Sai lầm 1: Quên $pre[0] = 0$
 - Sai lầm 2: Nhầm chỉ số 0-indexed và 1-indexed
 - Sai lầm 3: Tràn số khi tính tổng
 - Sai lầm 4: Difference array — quên +2 kích thước mảng



Tiếng Việt



Trong bài học này, chúng ta sẽ học một kỹ thuật vô cùng hữu ích và xuất hiện cực kỳ thường xuyên trong competitive programming: **Mảng tiền tố** (Prefix Sum). Kỹ thuật này biến bài toán tính tổng đoạn từ **chậm** thành **cực nhanh**.

1. Bài toán mở đầu: Truy vấn tổng đoạn

Cho mảng a gồm n số nguyên. Bạn cần trả lời q truy vấn, mỗi truy vấn cho hai số l và r , hỏi tổng $a[l] + a[l + 1] + \dots + a[r]$.

Ví dụ: $a = [3, 1, 4, 1, 5, 9, 2, 6]$

- Truy vấn $[2, 5]$: $1 + 4 + 1 + 5 = 11$
- Truy vấn $[1, 8]$: $3 + 1 + 4 + 1 + 5 + 9 + 2 + 6 = 31$
- Truy vấn $[4, 4]$: 1

Cách "thô": Duyệt từng phần tử

C++

Python

```
// Với mỗi truy vấn (l, r):  
long long sum = 0;  
for (int i = l; i <= r; i++) {  
    sum += a[i];  
}
```

```
# Với mỗi truy vấn (l, r):  
total = 0  
for i in range(l, r + 1):  
    total += a[i]
```

Mỗi truy vấn mất $O(n)$. Nếu $n = 10^5$ và $q = 10^5$, tổng cộng $O(n \times q) = O(10^{10}) \rightarrow$ **quá chậm!**

Câu hỏi: Có cách nào trả lời mỗi truy vấn trong $O(1)$ không?

Có! Đó chính là Prefix Sum.



Tiếng Việt



Định nghĩa

Cho mảng $a[1], a[2], \dots, a[n]$ (đánh số từ 1). Ta xây mảng pre (prefix sum) như sau:

$$pre[0] = 0$$

$$pre[1] = a[1]$$

$$pre[2] = a[1] + a[2]$$

$$pre[3] = a[1] + a[2] + a[3]$$

...

$$pre[i] = a[1] + a[2] + \dots + a[i] = pre[i-1] + a[i]$$

Nói cách khác, $pre[i]$ là **tổng của i phần tử đầu tiên** của mảng.

Ví dụ trực quan từng bước

Cho mảng $a = [3, 1, 4, 1, 5]$ (đánh số từ 1 đến 5).

Bước xây dựng:

$$pre[0] = 0$$

(quy ước)

$$pre[1] = pre[0] + a[1] = 0 + 3 = 3$$

(tổng 1 phần tử đầu)

$$pre[2] = pre[1] + a[2] = 3 + 1 = 4$$

(tổng 2 phần tử đầu)

$$pre[3] = pre[2] + a[3] = 4 + 4 = 8$$

(tổng 3 phần tử đầu)

$$pre[4] = pre[3] + a[4] = 8 + 1 = 9$$

(tổng 4 phần tử đầu)

$$pre[5] = pre[4] + a[5] = 9 + 5 = 14$$

(tổng 5 phần tử đầu)

Kết quả:

Chỉ số:	0	1	2	3	4	5
a:	---	3	1	4	1	5
pre:	0	3	4	8	9	14



Tiếng Việt



Tổng đoạn $[l, r] = pre[r] - pre[l - 1]$

Tại sao công thức này đúng?

$$pre[r] = a[1] + a[2] + \dots + a[l-1] + a[l] + \dots + a[r]$$

$$pre[l-1] = a[1] + a[2] + \dots + a[l-1]$$

$$pre[r] - pre[l-1] = a[l] + a[l+1] + \dots + a[r]$$

Ta "cắt bỏ" phần đầu mà ta không cần!

Kiểm tra lại với ví dụ

```
a = [3, 1, 4, 1, 5]
pre = [0, 3, 4, 8, 9, 14]
```

- Tổng $[2, 4] = pre[4] - pre[1] = 9 - 3 = 6 \checkmark$ (kiểm tra: $1 + 4 + 1 = 6$)
- Tổng $[1, 5] = pre[5] - pre[0] = 14 - 0 = 14 \checkmark$ (kiểm tra: $3 + 1 + 4 + 1 + 5 = 14$)
- Tổng $[3, 3] = pre[3] - pre[2] = 8 - 4 = 4 \checkmark$ (kiểm tra: $a[3] = 4$)
- Tổng $[1, 1] = pre[1] - pre[0] = 3 - 0 = 3 \checkmark$ (kiểm tra: $a[1] = 3$)

Tất cả đều đúng! Và mỗi truy vấn chỉ cần **1 phép trừ** — $O(1)$!

Code

C++

Python



```
using namespace std;

int main() {
    int n, q;
    cin >> n >> q;

    vector<long long> a(n + 1), pre(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    // Xây dựng mảng tiền tố – O(n)
    for (int i = 1; i <= n; i++) {
        pre[i] = pre[i - 1] + a[i];
    }

    // Trả lời q truy vấn – O(1) mỗi truy vấn
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << pre[r] - pre[l - 1] << "\n";
    }

    return 0;
}
```



Tiếng Việt



```

a = [0] + list(map(int, input().split()))
pre = [0] * (n + 1)

# Xây dựng mảng tiền tố – O(n)
for i in range(1, n + 1):
    pre[i] = pre[i - 1] + a[i]

# Trả lời q truy vấn – O(1) mỗi truy vấn
for _ in range(q):
    l, r = map(int, input().split())
    print(pre[r] - pre[l - 1])

```

Phân tích độ phức tạp:

- Tiền xử lý: $O(n)$
- Mỗi truy vấn: $O(1)$
- Tổng: $O(n + q)$ — nhanh gấp nhiều lần so với $O(n \times q)$!

3. Prefix Sum với mảng đánh số từ 0

Trong nhiều bài, mảng đánh số từ 0 ($a[0], a[1], \dots, a[n - 1]$). Khi đó ta vẫn xây prefix sum tương tự:

C++

Python

```

vector<long long> pre(n + 1, 0);
for (int i = 0; i < n; i++) {
    pre[i + 1] = pre[i] + a[i];
}

// Tổng đoạn a[l..r] (0-indexed):
long long sum_lr = pre[r + 1] - pre[l];

```



Tiếng Việt



```
for i in range(n):
    pre[i + 1] = pre[i] + a[i]

# Tổng đoạn a[l..r] (0-indexed):
sum_lr = pre[r + 1] - pre[l]
```

“

Lưu ý: Khi đánh số từ 0, công thức hơi khác một chút. Hãy cẩn thận và **luôn test với ví dụ nhỏ** để kiểm tra.

4. Ví dụ ứng dụng: Tìm đoạn con có tổng bằng S

Bài toán: Cho mảng a gồm n số nguyên **dương**. Đếm số đoạn con $[l, r]$ có tổng bằng S .

Cách giải: Tổng $[l, r] = pre[r] - pre[l - 1] = S$ nghĩa là $pre[l - 1] = pre[r] - S$.

Ta duyệt r từ 1 đến n , và đếm xem có bao nhiêu giá trị $pre[l - 1]$ (với $0 \leq l - 1 < r$) bằng $pre[r] - S$. Dùng `map` để lưu số lần xuất hiện:

C++ Python

```
map<long long, int> cnt;
cnt[0] = 1; // pre[0] = 0 xuất hiện 1 lần
long long pre = 0, ans = 0;
for (int i = 1; i <= n; i++) {
    pre += a[i];
    ans += cnt[pre - S];
    cnt[pre]++;
}
cout << ans << endl;
```



Tiếng Việt



```
cnt = defaultdict(int)
cnt[0] = 1 # pre[0] = 0 xuất hiện 1 lần
pre = 0
ans = 0

for i in range(1, n + 1):
    pre += a[i]
    ans += cnt[pre - S]
    cnt[pre] += 1

print(ans)
```

5. Mảng hiệu (Difference Array)

Bài toán mở đầu

Cho mảng a gồm n phần tử, ban đầu tất cả bằng 0. Thực hiện q phép biến đổi, mỗi phép cộng giá trị v vào tất cả phần tử trong đoạn $[l, r]$. Sau tất cả phép biến đổi, in mảng kết quả.

Cách "thô": Mỗi phép biến đổi, duyệt từ l đến r và cộng v :

C++ Python

```
for (int i = l; i <= r; i++) a[i] += v;
```

```
for i in range(l, r + 1):
    a[i] += v
```

Mỗi phép mất $O(n) \rightarrow$ tổng $O(n \times q) \rightarrow$ chậm!

Ý tưởng Difference Array

Thay vì cộng vào từng phần tử, ta **chỉ đánh dấu ở 2 vị trí**:

- Cộng v vào vị trí l (bắt đầu tác dụng)
- Trừ v ở vị trí $r + 1$ (hết tác dụng)



Tiếng Việt



Ví dụ trực quan

Mảng 5 phần tử, ban đầu toàn 0. Thực hiện 3 phép:

1. Cộng $+2$ vào đoạn $[2, 4]$
2. Cộng $+3$ vào đoạn $[1, 3]$
3. Cộng -1 vào đoạn $[3, 5]$

Mảng diff (mảng hiệu):

Chỉ số:	1	2	3	4	5	6
Ban đầu:	0	0	0	0	0	0

Phép 1: $[2, 4] +2 \rightarrow \text{diff}[2] += 2, \text{diff}[5] -= 2$

diff:	0	2	0	0	-2	0
-------	---	---	---	---	----	---

Phép 2: $[1, 3] +3 \rightarrow \text{diff}[1] += 3, \text{diff}[4] -= 3$

diff:	3	2	0	-3	-2	0
-------	---	---	---	----	----	---

Phép 3: $[3, 5] -1 \rightarrow \text{diff}[3] += (-1), \text{diff}[6] -= (-1)$

diff:	3	2	-1	-3	-2	1
-------	---	---	----	----	----	---

Tính prefix sum của diff:

Chỉ số:	1	2	3	4	5
diff:	3	2	-1	-3	-2
prefix sum:	3	5	4	1	-1

Kiểm tra bằng cách thủ công:

Chỉ số:	1	2	3	4	5	
Ban đầu:	0	0	0	0	0	
Phép 1:	0	2	2	2	0	(+2 ở $[2, 4]$)
Phép 2:	3	5	5	2	0	(+3 ở $[1, 3]$)
Phép 3:	3	5	4	1	-1	(-1 ở $[3, 5]$)



Code



Tiếng Việt

**C++** Python

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, q;
    cin >> n >> q;

    vector<long long> diff(n + 2, 0); // +2 để tránh truy cập ngoài mảng

    while (q--) {
        int l, r;
        long long v;
        cin >> l >> r >> v;
        diff[l] += v;
        diff[r + 1] -= v;
    }

    // Tính prefix sum để ra mảng kết quả
    for (int i = 1; i <= n; i++) {
        diff[i] += diff[i - 1];
        cout << diff[i] << " ";
    }
    cout << endl;

    return 0;
}
```



Tiếng Việt



```

diff = [0] * (n + 2) # +2 để tránh truy cập ngoài mảng

for _ in range(q):
    l, r, v = map(int, input().split())
    diff[l] += v
    diff[r + 1] -= v

# Tính prefix sum để ra mảng kết quả
for i in range(1, n + 1):
    diff[i] += diff[i - 1]

print(*diff[1:n + 1])

```

Phân tích:

- Mỗi phép biến đổi: $O(1)$ (chỉ cập nhật 2 vị trí)
- Tính kết quả cuối: $O(n)$
- Tổng: $O(n + q)$ — nhanh hơn rất nhiều so với $O(n \times q)$!

“

Mối liên hệ: Prefix sum và Difference array là **phép ngược** của nhau. Nếu *pre* là prefix sum của *a*, thì *a* chính là difference array của *pre*. Ngược lại, nếu *diff* là difference array, thì prefix sum của *diff* cho ta mảng gốc.

6. Tổng đoạn con lớn nhất (Kadane's Algorithm)**Bài toán**

Cho mảng *a* gồm *n* số nguyên (có thể âm). Tìm đoạn con liên tiếp $[l, r]$ ($1 \leq l \leq r \leq n$) có tổng lớn nhất.

Ví dụ: $a = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$

Đoạn con có tổng lớn nhất là $[4, -1, 2, 1]$ với tổng = 6.

Cách "thô": Thử mọi đoạn con — $O(n^2)$



Tiếng Việt



```

long long max_sum = a[1];
for (int l = 1; l <= n; l++) {
    long long sum = 0;
    for (int r = l; r <= n; r++) {
        sum += a[r];
        max_sum = max(max_sum, sum);
    }
}

```

```

max_sum = a[1]
for l in range(1, n + 1):
    total = 0
    for r in range(l, n + 1):
        total += a[r]
        max_sum = max(max_sum, total)

```

Với $n = 10^5$, $O(n^2) = 10^{10} \rightarrow$ quá chậm!

Kadane's Algorithm — $O(n)$

Ý tưởng chính: Duyệt mảng từ trái sang phải, duy trì biến `cur` = tổng đoạn con kết thúc tại vị trí hiện tại.

Tại mỗi vị trí i , ta có 2 lựa chọn:

1. **Nối tiếp** đoạn cũ: `cur + a[i]` (mở rộng đoạn hiện tại)
2. **Bắt đầu mới** từ $a[i]$: chỉ lấy `a[i]` (bỏ đoạn cũ)

Ta chọn cách nào cho tổng lớn hơn. Nếu `cur < 0` thì bắt đầu mới luôn tốt hơn (vì cộng thêm số âm chỉ làm giảm tổng).

Ví dụ trực quan từng bước

Mảng: $a = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$



Tiếng Việt



$cur = 0 + (-2) = -2 \rightarrow cur < 0 \rightarrow \text{reset } cur = 0$

$max_sum = \max(-INF, -2) = -2$

(Ghi chú: ta vẫn cập nhật max_sum TRƯỚC khi reset)

$i = 2: a[i] = 1$

$cur = 0 + 1 = 1 \rightarrow cur \geq 0 \rightarrow \text{giữ}$

$max_sum = \max(-2, 1) = 1$

$i = 3: a[i] = -3$

$cur = 1 + (-3) = -2 \rightarrow cur < 0 \rightarrow \text{reset } cur = 0$

$max_sum = \max(1, -2) = 1$

$i = 4: a[i] = 4$

$cur = 0 + 4 = 4 \rightarrow cur \geq 0 \rightarrow \text{giữ}$

$max_sum = \max(1, 4) = 4$

$i = 5: a[i] = -1$

$cur = 4 + (-1) = 3 \rightarrow cur \geq 0 \rightarrow \text{giữ}$

$max_sum = \max(4, 3) = 4$

$i = 6: a[i] = 2$

$cur = 3 + 2 = 5 \rightarrow cur \geq 0 \rightarrow \text{giữ}$

$max_sum = \max(4, 5) = 5$

$i = 7: a[i] = 1$

$cur = 5 + 1 = 6 \rightarrow cur \geq 0 \rightarrow \text{giữ}$

$max_sum = \max(5, 6) = 6$

$i = 8: a[i] = -5$

$cur = 6 + (-5) = 1 \rightarrow cur \geq 0 \rightarrow \text{giữ}$

$max_sum = \max(6, 1) = 6$

$i = 9: a[i] = 4$

$cur = 1 + 4 = 5 \rightarrow cur \geq 0 \rightarrow \text{giữ}$

$max_sum = \max(6, 5) = 6$



Tiếng Việt



Code

C++

Python

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;

    long long max_sum = a[0], cur = 0;
    for (int i = 0; i < n; i++) {
        cur += a[i];
        max_sum = max(max_sum, cur);
        if (cur < 0) cur = 0;
    }

    cout << max_sum << endl;
    return 0;
}
```



Tiếng Việt



```

a = list(map(int, input().split()))

max_sum = a[0]
cur = 0

for x in a:
    cur += x
    max_sum = max(max_sum, cur)
    if cur < 0:
        cur = 0

print(max_sum)

```

Tại sao khởi tạo `max_sum = a[0]` ? Vì nếu tất cả phần tử đều âm (ví dụ $[-3, -2, -1]$), ta vẫn phải chọn ít nhất 1 phần tử. `a[0]` đảm bảo đáp án không phải 0 (mà là số âm lớn nhất).

Tại sao reset `cur = 0` khi `cur < 0` ? Vì một đoạn có tổng âm sẽ chỉ làm giảm tổng của bất kỳ đoạn nào nối tiếp sau nó. Bắt đầu mới từ đầu luôn tốt hơn.

7. Prefix Sum 2D (mở rộng)

Kỹ thuật prefix sum có thể mở rộng cho bảng 2 chiều. Cho bảng $a[1..n][1..m]$, ta xây:

$pre[i][j]$ = tổng các phần tử $a[x][y]$ với $1 \leq x \leq i$ và $1 \leq y \leq j$

Công thức xây dựng:

$pre[i][j] = pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1] + a[i][j]$

Tổng hình chữ nhật $[r_1, c_1]$ đến $[r_2, c_2]$:

$sum = pre[r_2][c_2] - pre[r_1-1][c_2] - pre[r_2][c_1-1] + pre[r_1-1][c_1-1]$



Tiếng Việt



8. Sai lầm thường gặp

Sai lầm 1: Quên `pre[0] = 0`

C++

Python

```
// SAI – nếu không có pre[0] = 0:
pre[1] = a[1]; // OK
// Nhưng tổng [1, r] = pre[r] - pre[0] sẽ sai nếu pre[0] chưa được gán!

// ĐÚNG:
vector<long long> pre(n + 1, 0); // pre[0] = 0 tự động
for (int i = 1; i <= n; i++)
    pre[i] = pre[i - 1] + a[i];
```

```
# SAI – nếu không có pre[0] = 0:
# tổng [1, r] = pre[r] - pre[0] sẽ sai nếu pre[0] chưa được gán!

# ĐÚNG:
pre = [0] * (n + 1)
for i in range(1, n + 1):
    pre[i] = pre[i - 1] + a[i]
```

Sai lầm 2: Nhầm chỉ số 0-indexed và 1-indexed

Nếu mảng đánh số từ 0 ($a[0], \dots, a[n - 1]$) mà bạn dùng công thức 1-indexed, kết quả sẽ lệch 1.

Hãy thống nhất cách đánh số và **luôn test với ví dụ nhỏ**.

Sai lầm 3: Tràn số khi tính tổng

Nếu $n = 10^5$ và mỗi $a[i]$ có thể đến 10^9 , thì prefix sum có thể đến 10^{14} → **phải dùng** `long long` !

C++

Python



Tiếng Việt



```
vector<int> pre(n + 1, 0); // int chỉ chứa đến  $\sim 2 \times 10^9$ 

// ĐÚNG:
vector<long long> pre(n + 1, 0);
```

Python int không bị tràn số theo kiểu int 32-bit,
nhưng trong C++ thì phải dùng long long.

Sai lầm 4: Difference array — quên **+2** kích thước mảng

Khi cộng giá trị vào đoạn $[l, r]$, ta cần truy cập `diff[r + 1]`. Nếu $r = n$, ta truy cập `diff[n + 1]`. Vì vậy mảng diff cần kích thước ít nhất $n + 2$.

C++

Python

```
// SAI:
vector<long long> diff(n + 1, 0); // diff[n+1] sẽ truy cập ngoài mảng!

// ĐÚNG:
vector<long long> diff(n + 2, 0); // an toàn
```

```
# SAI:
diff = [0] * (n + 1) # diff[n+1] sẽ vượt chỉ số

# ĐÚNG:
diff = [0] * (n + 2)
```

9. Tóm tắt

KỸ THUẬT	BÀI TOÁN	TIỀN XỬ LÝ	MỖI TRUY VẤN/PHÉP BIẾN ĐỔI
Prefix Sum	Tổng đoạn $[l, r]$	$O(n)$	$O(1)$
Difference Array	Cộng v vào đoạn $[l, r]$	—	$O(1)$ mỗi phép, $O(n)$ tổng kết



Tiếng Việt



Kadane's Algorithm	Tổng đoạn con lớn nhất	—	$O(n)$
--------------------	------------------------	---	--------

Mẹo ghi nhớ:

- Prefix sum = "tổng tích lũy" — dùng khi cần hỏi tổng nhiều lần
- Difference array = "ngược lại" của prefix sum — dùng khi cần cập nhật nhiều lần
- Kadane = "đi từ trái sang phải, giữ lại nếu tốt, bỏ đi nếu xấu"

Chúc các bạn luyện tập tốt! Prefix sum là kỹ thuật nền tảng mà bạn sẽ dùng đi dùng lại trong suốt hành trình CP. Hãy nắm thật vững trước khi sang bài tiếp theo nhé!

Bài tập

Ambatukam	0 / 10
CSES - Static Range Sum Queries Truy vấn tổng mảng tĩnh	0 / 10
Tổng k số	0 / 15
Dải số	0 / 15
CSES - Restaurant Customers Khách nhà hàng	0 / 15
MINI CANDY	0 / 15
CSES - Maximum Subarray Sum Tổng đoạn con lớn nhất	0 / 20
Tổng dãy con	0 / 20
Tổng liên tiếp không quá t	0 / 20
CSES - Subarray Sums I Tổng đoạn con I	0 / 20
Tổng bằng 0	0 / 25
Bài dễ (DHBB 2021)	0 / 25